AEGIS

AI Energy Geopolitical Intelligence System

TECHNICAL SPECIFICATION & DOCUMENTATION

Version: 2.0.0 (FastAPI + React Redesign)

Status: Production Ready

Target Environment: Enterprise Local / Cloud

Designed for India's Crude Oil Supply Chain Monitoring,
Strategic Petroleum Reserve Optimization, & Geopolitical Shock Modelling.

1. Project Overview & Problem Statement

India imports over 85% of its crude oil requirements, making its energy security highly sensitive to geopolitical tensions, maritime chokepoint blocks, and regional supply chain bottlenecks. Standard dashboards offer passive telemetry display, lacking the capabilities to simulate what-if shock scenarios, re-route supply paths in real-time, or optimize reserve releases dynamically.

AEGIS (AI Energy Geopolitical Intelligence System) solves this critical security gap by providing a comprehensive, full-stack mission-control console. It integrates real-time telemetry from multiple maritime corridors, parses geopolitical news signals, runs mathematical simulations for supply chain disruptions, recommends alternative procurement plans, and schedules Strategic Petroleum Reserve (SPR) drawdowns.

2. Overall Architecture & System Workflow

AEGIS is organized as a contract-first monorepo. The core system architecture consists of a React client frontend, a FastAPI Python backend server, and a PostgreSQL database. The API schema is defined in an OpenAPI specification, serving as the single source of truth that drives client code generation and backend routing.

1. Data Ingestion: News feeds and maritime sensors push indicators into the database.
2. API Gateway: FastAPI processes request queries, implements validation via Python schemas, and serves ORM data.
3. Client UI: The React frontend fetches data using React Query and renders the Crimson Command dashboard.

3. End-to-End Operation & Telemetry Loop

Telemetry ingestion starts when geopolitical indicators or port congestions are updated in the database. The Python backend translates these changes into risk calculations. Upon user request, a background simulation (e.g. Strait of Hormuz blockage) recalculates supplier shares, updates projected refinery run rates, and estimates national GDP impact. The results are pushed instantly to the client UI using Vite Hot Module Replacement (HMR) and React Query buffer cache updates, rendering the complete situation map.

4. Feature-by-Feature Module Breakdown

Module 1: Mission Control (Overview)

The Command Overview page provides a high-level briefing. It features a system status banner (NOMINAL / ELEVATED / CRITICAL), circular gauges representing overall risk score and reserve cover, and active what-if simulation builder triggers.

Module 2: Risk Intelligence

Displays real-time threat indices across five primary shipping corridors (Strait of Hormuz, Red Sea, Malacca Strait, Cape of Good Hope, Persian Gulf). Includes a live news ticker stream and a supplier vulnerability matrix.

Module 3: Scenario Simulation

Allows operators to model specific disruptions (e.g. regional strikes, corridor closures). Displays a 2x2 comparison grid tracking Refinery Run Rate, Fuel Price, Power Stress, and GDP impact.

Module 4: Procurement Intelligence

Runs an AI optimizer to rank alternative sourcing contracts based on spot price, tanker availability, port congestion, and grade compatibility. Includes structured reasoning text details.

Module 5: Reserve Optimizer

Simulates drawdown lifespans for Strategic Petroleum Reserves. A manual release slider models cost impacts and outputs a projected inventory depletion area chart.

Module 6: Digital Twin Map

Features a full-screen equirectangular SVG projection layering weather wind lines, satellite grid scans, and active vessels traffic. Click targets trigger node and corridor telemetry feeds.

Module 7: Analytics & Reports

Generates historical risk charts, route efficiency bars, supplier radar portfolios, and supports report downloads.

Module 8: AI Command Center

Provides a conversational terminal that answers natural language diagnostics questions using mapped database schemas.

Module 9: Settings & Diagnostics

Manages general preferences, alerts thresholds, and connected database schemas (with tables records count) along with a live diagnostic log output.

5. Frontend & Backend Architecture

The monorepo structure separating frontend, backend, and schemas is detailed below:

Workspace Area	Language	Path / Folder	Responsibilities
aegis-client	TSX / CSS	artifacts/aegis/src	Vite build, layout, UI, wouter routes
api-server	Python	artifacts/api-server/src	FastAPI endpoints, SQL Alchemy ORM, seed.py
api-spec	OpenAPI	lib/api-spec	OpenAPI yaml contract definition
api-zod	TypeScript	lib/api-zod	Autogenerated Zod schema validation files
api-client-react	TypeScript	lib/api-client-react	React Query generated data-hooks

6. AI Components & Cognitive Integrations

AI components in AEGIS are integrated directly into the database layers and backend route controllers:
1. Natural Language Diagnostics (AI Command): The conversational terminal translates schema variables into text replies.
2. Contract Sourcing Reasoning: The Procurement Optimizer populates a 'reasoning' column in the recommendations table using LLM-style decision matrices based on compatibility and congestion.
3. Disruption Impact Modelling: Scenario routes simulate output fluctuations on refinery runs.

7. API Reference & Specifications

All communications between the React client and the FastAPI backend are mapped via the following endpoints:

Method	Endpoint	Data Response	Description
GET	/api/overview/summary	OverviewSummary	Command health state and core sparklines
GET	/api/risk/scores	List[RiskScore]	Corridor risk dial ratings and trends
GET	/api/risk/signals	List[NewsSignal]	Geopolitical stream warnings
GET	/api/risk/suppliers	List[SupplierRisk]	Supplier contract shares and routes
GET	/api/procurement/recs	List[ProcurementRec]	AI-ranked alternative sourcing plans
GET	/api/reserve/status	ReserveStatus	SPR inventory cover and recommended actions
GET	/api/digital-twin/nodes	List[DigitalTwinNode]	Coordinates and metadata of GIS targets
GET	/api/system/config	SystemConfigResponse	Database connections and API health status

8. Database Schemas & Mappings

The PostgreSQL database is organized into 7 tables mapped via SQLAlchemy models in src/models.py:

Table Name	Primary Key	Key Attributes	Usage
risk_scores	id (Integer)	corridor, score, level, trend	Risk Intelligence dials
news_signals	id (Integer)	headline, source, corridor, severity	Live operations feed
supplier_risks	id (Integer)	supplier, country, share, risk_level	Vulnerability matrices
procurement_recommendations	id (Integer)	rank, spot_price, overall_score, reasoning	Contract sourcing planner
reserve_status	id (Integer)	current_days, capacity_days, drawdown_rate	Reserve Optimization status
digital_twin_nodes	id (String)	name, type, lat, lon, risk_level	Coordinates for GIS map
shipping_corridors	id (String)	from, to, waypoints (JSONB)	Shipping path polylines

9. Data Ingestion & Flow Map

1. Geopolitical sensors publish updates -> 2. Python backend calculates corridor threat indices -> 3. Database updates transaction status -> 4. React query invalidates active key cache -> 5. Frontend pulls updated JSON model -> 6. Recharts components redraw layout canvas.

10. System Dependencies & Third-Party Libraries

1. Python Backend: fastapi, uvicorn, sqlalchemy, psycopg2-binary, pydantic, python-dotenv.
2. React Frontend: tailwindcss (v4), framer-motion, recharts, wouter, @tanstack/react-query, lucide-react.

11. Security & Compliance Settings

1. Connection String Safety: Database URLs loaded via dotenv variables.
2. CORS Middleware: Standard FastAPI headers restrict unauthorized API calls.
3. Secrets Scanning: Removed hardcoded placeholders (e.g. Slack Webhooks) to prevent push blocks.

12. Deployment & Setup Guidelines

To deploy, run 'python3 -m pip install -r requirements.txt' in the api-server directory. Recreate database tables using 'python3 src/seed.py', and launch the server using 'python3 -m uvicorn src.main:app --port 8080'.

13. Codebase Directory Tree Structure

```
aegis-production/
 +-- artifacts/
 |   +-- aegis/             # Frontend workspace
 |   |   +-- src/pages/     # Workspaces (overview, risk, settings, etc.)
 |   |   +-- src/components/ # layout.tsx, scenario-video.tsx
 |   +-- api-server/        # Backend Python workspace
 |       +-- src/
 |           +-- routes/    # Overview, risk, digital twin, etc.
 |           +-- models.py  # SQLAlchemy DB definitions
 |           +-- seed.py    # Python DB seeder script
 +-- lib/                   # Shared API client and schema libraries
```

14. Technology Stack Justifications

1. Python (FastAPI): High performance, asynchronous request handling, and compatibility with AI models.
2. React + Tailwind CSS 4: Premium visual aesthetics, high performance rendering, and rich UI elements.
3. PostgreSQL + SQLAlchemy: Secure relational mapping and flexibility for complex data structure queries.

15. Performance & Scalability Considerations

1. Database Connection Pooling: Configured engine pool size to 10 and max overflow to 20.
2. Ticker Ribbon Render: Implemented a pure CSS keyframe marquee for infinite scroll to prevent CPU spikes.
3. React Query caching: Invalidates data triggers only when manual refresh is clicked.

16. Future Scope & Enhancements

1. Live Map Integrations: Layering Google Maps GIS coordinates over the SVG digital twin model.
2. Machine Learning: Integrating predictive threat scores based on live news stream sentiments.